

DA2304 Assignment 2 – Report

Athreya Ganesh

DA24B002

Extra Credits

[One Of These Things First - Nick Drake](#)

Part 1: The VM Translator

Exercise 1: Building the Hack VM Translator

I have built a Hack VM Translator which takes in .vm files and converts them to .asm files of the same name, and saves them along with the .vm file. It consists of three separate files: `codewriter.py`, `parser.py`, and `translate.py`.

parser.py

This uses regex to read every line in the .vm file, take each individual token in it and turn it to an array. We know there can be at most 3 tokens per line in Hack VM, so we store these tuples in a class object `self.file` after stripping away whitespaces and comments.

codewriter.py

This forms the heart of the translator. I first form a segment map with local, argument, this, that and their respective memory address terms. I have separate sections for each type of command, respectively arithmetic, push/pop, label, goto, if, call, return, function. I write these translations in a file and save it.

translate.py

This is the file which the user directly interacts with. To translate, we use the command:

```
python translate.py <file.vm>
```

And the translated file gets saved in the same path as the vm file.

Part 2: Matrix Operations (MAC)

Exercise 1: The MAC Operation Architecture

Sanity Checking

We take a specific memory address which we take to be an indicator to whether the matrices can be multiplied, and set it to +1 or -1 depending on whether the tests pass or fail respectively.

PSEUDOCODE:

```
function throw_error(W_rows, W_cols, X_rows, X_cols, B_rows, B_cols):  
    if W_cols != X_rows:  
        set memory[ERROR_LOC] = -1  
        return False  
  
    if W_rows != B_rows:  
        set memory[ERROR_LOC] = -1  
        return False  
  
    set memory[ERROR_LOC] = 1  
    return True
```

VM CODE:

```
// argument 0: W_rows  
// argument 1: W_cols  
// argument 2: X_rows  
// argument 3: X_cols  
// argument 4: B_rows  
// argument 5: B_cols  
function sanity_check 0  
    push argument 1  
    push argument 2  
    eq  
    not  
    if-goto FAIL  
  
    push argument 0  
    push argument 4  
    eq  
    not  
    if-goto FAIL  
  
    push argument 5  
    push constant 1  
    eq  
    not  
    if-goto FAIL  
label PASS
```

```
push constant 5000
pop pointer 1
push constant 1
pop that 0
```

```
push constant 1
return
```

label FAIL

```
push constant 5000
pop pointer 1
push constant 1
neg
pop that 0
```

```
push constant 0
return
```

Exercise 2: Architectural Analysis

1. Stack Usage:

- MATMUL: The function is only called once in the entire program. Hence, saving the LCL, ARG, THIS, THAT pointers happens only once. The stack has a low overhead cause of this, as it only has to store the arguments and the local variables for the triple nested loop.
- ROWXCOL: This approach moves the nested loop to the caller function, which calls ROWXCOL for each element Y_{ij} . For an $m \times n$ matrix, the function ROWXCOL is called mn times, which uses up a lot more assembly instructions than a single triple nested loop in MATMUL, as it has to save the LCL, ARG, THIS, THAT memory pointers and access them over and over again. This stack has a high overhead.

2. Segment Usage:

- MATMUL: This heavily depends on the temporary memory registers (temp 0 to temp 3) to store the loop iterators and the sum. The drawback is, if any function modifies temp 0, then the entire logic breaks down.
- ROWXCOL: This uses local variable to store the sum. This is safe from the risk from using temp.

3. Optimization (Approach A):

Since Hack ALU does not include a multiplier, the multiplication process is much slower than it should be, hence a multiplication hardware feature can be added to the ALU. Software wise this would make finding $W[i][j]$ so much easier. I could still store $(i * N)$ into a memory address and increment it by 1 though.

4. Optimization (Approach B):

This is strongly disadvantaged by the stack overhead. The VM Translator could copypaste the code from ROWXCOL to where it is called to make things much simpler and avoid the stack overhead.